

REINFORCE PRO MAX: CAUSAL PREFIX FILTERING AND ADAPTIVE ADVANTAGE NORMALIZATION FOR CRITIC-FREE LLM REINFORCEMENT LEARNING

Anonymous authors

Paper under double-blind review

ABSTRACT

Critic-free reinforcement learning for language models must control both token-mass imbalance in sampled advantages and the mismatch between rollout and training policies. We present REINFORCE Pro Max, which combines leave-one-out advantages, sign-preserving asymmetric normalization, and a prefix-cumulative importance-sampling gate. The gate is computed from the cached rollout/snapshot ratio but gradients flow only through the current/snapshot PPO ratio. On a finite autoregressive tree, we prove an exact decomposition of the implemented standard-PPO objective into the raw surrogate, gate and normalization corrections, and a nonnegative clipping deduction. A finite-tree adaptive bound then gives a conditional sufficient certificate for reward improvement. The mathematical statements and their finite-tree proofs are checked in Lean; the formalization does not claim to verify floating-point neural-network training. The current artifact contains structural synthetic checks only and does not report an LLM benchmark.

1 PROBLEM AND SCOPE

Critic-free objectives such as RLOO and GRPO are widely used for reasoning tasks (Ahmadian et al., 2024; Shao et al., 2024), while PPO remains a standard clipped policy-gradient baseline (Schulman et al., 2017). Recent long-horizon trust-region analyses motivate retaining per-position causal structure (Li et al., 2025). LLM-RL rollouts are generated by π_{roll} while the actor is updated from a frozen snapshot π_{old} . For response $y = (y_1, \dots, y_T)$, write $\rho_t = \pi_{\theta}(y_t | c_t) / \pi_{\text{roll}}(y_t | c_t)$ and $w_t = \pi_{\text{old}}(y_t | c_t) / \pi_{\text{roll}}(y_t | c_t)$. The analyzed configuration is standard PPO (`policy_loss_type=ppo`), `vllm_is_correction_type=reinforce_pro`, no dual clip, no KL or entropy term, and sparse terminal reward with zero KL coefficient. These restrictions are essential: optional dual clipping and auxiliary losses change the objective.

We study a finite response alphabet and finite horizon with mutual support, bounded reward $|R| \leq R_{\text{max}}$, and explicitly stated trust-region or moment assumptions. The results are conditional certificates, not guarantees that an arbitrary trained neural policy satisfies those assumptions.

2 METHOD

For a prompt group of n responses, the raw leave-one-out signal is $A_i = R_i - (n-1)^{-1} \sum_{j \neq i} R_j$. After token expansion, positive and negative active tokens need not have equal mass. Let P, N be the positive and negative token sets. The ideal ProMax factors are chosen so that $\sum_P \alpha A + \sum_N \beta A = 0$ and $|P|^{-1}|N|^{-1}$ -free second-moment normalization equals one on nonzero tokens, with $\alpha, \beta > 0$. Thus signs and within-sign ordering are preserved. The implementation adds an epsilon, cap, clamp, and fallback; the formal result treats their moment effects as explicit residuals.

For each active token, the prefix proxy is

$$M_t^{\text{pre}} = \mathbf{1} \left\{ \ell_- \leq \frac{1}{t} \sum_{s \leq t} \log w_s \leq \ell_+ \right\}, \quad w_t = \exp(\log \pi_{\text{old}} - \log \pi_{\text{roll}}).$$

The implemented loss is

$$\mathcal{L} = \text{MaskedMean}_t \left[M_t^{\text{pre}} \text{stopgrad}(w_t) \left(-\min(r_t \hat{A}_t, \text{clip}(r_t) \hat{A}_t) \right) \right], \quad r_t = \frac{\pi_\theta}{\pi_{\text{old}}}.$$

The prefix gate, cached importance coefficient, action mask, and normalized advantage are detached. Only r_t carries policy gradient.

3 FORMAL RESULTS

Let k index represented response/token positions, let $\omega_k \geq 0$ be the chosen reduction weight, M_k the gate, r_k and w_k the two probability ratios, A_k the raw signal, and H_k the implemented normalized signal. For standard PPO define

$$U = \sum_k \omega_k r_k w_k A_k, \quad G = \sum_k \omega_k r_k w_k (1 - M_k) A_k, \quad N = \sum_k \omega_k r_k w_k M_k (H_k - A_k),$$

and

$$C = \sum_k \omega_k M_k w_k (r_k H_k - \min(r_k H_k, \text{clip}(r_k) H_k)).$$

Then the finite-batch identity is

$$-\mathcal{L} = U - G + N - C, \quad C \geq 0.$$

The gate and normalization terms are signed corrections. The nonnegativity of C is a standard-PPO fact and does not extend to dual clip; a Lean-checked counterexample is included in `formal/ProMaxObjective.lean`.

For a finite autoregressive tree, if every rollout-reachable history of length less than T satisfies either a local TV bound ε or a local reverse-KL bound δ , the future continuation discrepancy after history length t is bounded by

$$\min\{1, (T - t)\varepsilon, \sqrt{(T - t)\delta/2}\}.$$

With bounded rewards, summing these continuation errors bounds the difference between the raw surrogate and the true reward change. Consequently, a corrected empirical ProMax gain exceeding the explicit gate, normalization, clipping, sampling, and adaptive remainder terms is sufficient for positive true reward gain. The statement is for independent evaluation blocks and a fixed finite candidate family, or a pre-specified class with a fixed finite cover.

The finite-tree identities, adaptive recursion, RLOO moments, reduction bounds, and binary-length covariance conditions are checked by the Lean files listed in `formal/PROOF_AUDIT.md`. The audit currently covers 31 labeled mathematical environments and 911 project theorems.

For binary rewards, the ideal normalization has two class amplitudes whose ratio is the ratio of class-conditional mean response lengths. With the actual epsilon stabilizer, cap, clamp, fallback, and token denominator, the complete on-policy update remains a positive multiple of the success-probability gradient when active length depends only on the reward class and both classes have positive mass. If length also varies within a reward class, the update decomposes into this class multiplier plus a within-class covariance error; a strict local-improvement conclusion then requires an explicit covariance margin. The theorem does not claim that arbitrary content-dependent lengths or an arbitrary Adam step improve reward.

4 IMPLEMENTATION CORRESPONDENCE

The code path uses active-token masking and distinguishes the differentiable PPO ratio from the detached rollout correction. Standard PPO is fixed to the configuration above; GSPO, dual clip, KL loss, entropy loss, MoE auxiliary loss, reward shaping, and dynamic group filtering require separate objective terms and are outside the stated certificate. Variable-length dynamic microbatch response-count weighting does not automatically equal a global token mean, so the population reduction must be reported explicitly.

5 EVALUATION STATUS AND LIMITATIONS

The current repository includes deterministic synthetic structural checks for normalization and prefix masking. It does not contain matched LLM training, multiple seeds, an independent AIME/MATH-style test set, or a benchmark claim. The formal proofs use exact real arithmetic on finite models; they do not formalize floating-point overflow, GPU kernels, Adam, Ray/vLLM execution, or the assumptions of a general neural policy. These are required before claiming empirical ICLR readiness.

AI USE STATEMENT

Generative AI tools were used to assist with code navigation, proof-audit organization, and language editing. All mathematical statements, source changes, and reported checks were reviewed against the repository and verified by the authors; no generated text is treated as empirical evidence. The authors take responsibility for the final content.

REPRODUCIBILITY STATEMENT

The repository contains the implementation, the exact finite-tree Lean development, the proof audit, and deterministic structural tests. The current artifact does not report GPU training or benchmark results; those experiments remain future work.

ETHICS STATEMENT

This work uses no human subjects or private user data.

REFERENCES

- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.
- Yingru Li, Jiakai Liu, Jiawei Xu, Yuxuan Tong, Ziniu Li, Qian Liu, and Baoxiang Wang. Trust region masking for long-horizon LLM reinforcement learning, 2025.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

A FORMALIZATION AND PROOF COVERAGE

The accompanying Lean project is built with the repository’s pinned toolchain and an explicit axiom audit. The audit records 31 labeled theorem, proposition, corollary, and lemma environments in the paper and maps each to a checked Lean declaration. The latest clean build checked 911 project theorems and allowed only `propext`, `Classical.choice`, and `Quot.sound`. This is a finite exact-real formalization; it does not certify floating-point kernels, Ray/vLLM scheduling, optimizer dynamics, or neural-network assumptions.

The implementation theorem is restricted to standard PPO with `vllm.is.correction.type=reinforce.pro`, no dual clip, no KL or entropy loss, zero auxiliary-loss coefficient, sparse terminal reward, and zero KL reward coefficient. The Lean counterexample in `formal/ProMaxObjective.lean` shows why the nonnegative clipping deduction cannot be extended to dual clipping. The finite-tree adaptive result assumes mutual support, bounded rewards, and the stated local TV or reverse-KL bounds. The binary-length result additionally requires positive mass in both reward classes; content-dependent within-class length variation is covered only with the stated covariance margin.

B REPRODUCIBILITY MATERIALS

The source tree contains the implementation changes in `openrlhf/models/loss.py` and `openrlhf/trainer/ppo_utils/experience_maker.py`, the formal development under `formal/`, and deterministic CPU checks under `reinforce_pro.max/`. The checks cover the policy-loss contract, prefix masking, normalization safeguards, and structural examples. No GPU is available in the current build environment, so no LLM training, multi-seed comparison, or AIME/MATH-style benchmark result is reported here.